

- Solution Architect — Detailed Interview Questionnaire & Answers
 - Part 1: AWS Architecture, Compute & Networking
 - Section 1: AWS Solution Architecture — Fundamentals
 - Q1. What does a Solution Architect do, and how do you differentiate between a Solution Architect and a Software Architect?
 - Q2. Walk us through how you approach designing a solution for a new client requirement.
 - Q3. What is the AWS Well-Architected Framework? Explain each pillar with a real-world example.
 - Pillar 1: Operational Excellence
 - Pillar 2: Security
 - Pillar 3: Reliability
 - Pillar 4: Performance Efficiency
 - Pillar 5: Cost Optimization
 - Pillar 6: Sustainability
 - Q4. How do you conduct an AWS Well-Architected Review for an existing workload?
 - Section 2: AWS Compute Services
 - Q5. Compare EC2, ECS (Fargate), EKS, and Lambda. When would you choose each?
 - Q6. Explain Auto Scaling strategies in AWS. How would you design auto-scaling for a web application?
 - Q7. What is AWS Lambda? Explain cold starts, concurrency, and when Lambda is NOT the right choice.
 - Q8. Explain the difference between ECS and EKS. When would you recommend EKS over ECS?
 - Section 3: AWS Networking
 - Q9. Design a production VPC architecture for a 3-tier web application. Explain each component.
 - Q10. Explain VPC Endpoints — Interface vs Gateway. Why are they important?
 - Q11. Explain Route53 routing policies and when to use each.
 - Q12. What is CloudFront and how does it integrate with your architecture?
 - Q13. Explain API Gateway — REST vs HTTP vs WebSocket. When would you use each?

Solution Architect — Detailed Interview Questionnaire & Answers

Part 1: AWS Architecture, Compute & Networking

Profile Context: 12-15 years IT experience | Python (testing) | Web Architecture Design | AWS Cloud | Docker/K8s | DevOps/CI-CD | AI Project Exposure | No mobile/IoT/Azure/GCP/development

Section 1: AWS Solution Architecture — Fundamentals

Q1. What does a Solution Architect do, and how do you differentiate between a Solution Architect and a Software Architect?

Answer:

A **Solution Architect** operates at the intersection of business and technology. The role involves:

- **Understanding business requirements** and translating them into technical solutions
- **Evaluating trade-offs** between cost, performance, security, and operational complexity
- **Creating architecture blueprints** — High-Level Design (HLD) and Low-Level Design (LLD) documents
- **Guiding development teams** with architectural decisions, design patterns, and technology stack recommendations

- **Ensuring non-functional requirements** (NFRs) like scalability, availability, disaster recovery, and compliance are baked into the design

Key Difference:

Aspect	Solution Architect	Software Architect
Scope	End-to-end system design across infrastructure, services, and integrations	Application-level design patterns, code structure
Focus	"What cloud services, how they connect, what trade-offs"	"How the code is organized, what frameworks, what patterns"
Deliverables	Architecture diagrams, HLD/LLD, cost estimates, migration plans	Class diagrams, API contracts, module design
Tools	AWS Console, Terraform, draw.io, cost calculators	IDE, UML tools, code review

In my experience, I've operated as a Solution Architect by designing **web application architectures** on AWS — defining the VPC topology, service selection (ECS vs. Lambda vs. EKS), security boundaries, and CI/CD pipelines, while the development team focuses on the application code.

Q2. Walk us through how you approach designing a solution for a new client requirement.

Answer:

I follow a structured **6-phase approach**:

Phase 1 — Discovery & Requirements Gathering

- Conduct stakeholder interviews to understand business goals, pain points, and constraints
- Document functional requirements (FRs) and non-functional requirements (NFRs)
- Identify compliance requirements (GDPR, SOC2, HIPAA, PCI-DSS)

- Understand existing systems and integration points

Phase 2 — Current State Assessment

- Map existing architecture ("as-is" state)
- Identify technical debt, bottlenecks, and single points of failure
- Evaluate team skill sets and organizational constraints

Phase 3 — Architecture Design

- Create the target state architecture ("to-be")
- Produce **High-Level Design (HLD)**: System context diagram, component diagram, data flow diagram
- Produce **Low-Level Design (LLD)**: Network topology, security groups, IAM policies, service configurations
- Define the **technology stack** with justification for each choice

Phase 4 — Trade-off Analysis & Decision Records

- Document Architecture Decision Records (ADRs) for each major decision
- Present cost analysis using AWS Pricing Calculator
- Conduct risk assessment with mitigation strategies

Phase 5 — Proof of Concept (PoC)

- Build a minimal PoC to validate critical assumptions
- Load test to validate NFRs (latency, throughput, failover)

Phase 6 — Implementation Roadmap

- Break down into sprints/phases with clear milestones
- Define success criteria and acceptance tests
- Establish monitoring and alerting strategy from Day 1

Q3. What is the AWS Well-Architected Framework? Explain each pillar with a real-world example.

Answer:

The AWS Well-Architected Framework provides **6 pillars** (updated from original 5) to evaluate and improve cloud architectures:

Pillar 1: Operational Excellence

Focus: Run and monitor systems to deliver business value; continuously improve processes

Key Practices:

- Infrastructure as Code (Terraform/CloudFormation)
- Automated deployments via CI/CD (GitLab CI + ArgoCD)
- Runbooks and playbooks for incident response
- Observability: CloudWatch dashboards, X-Ray tracing, structured logging

Example: In my EKS project, I implemented GitOps with ArgoCD where every infrastructure change goes through a Git PR → review → merge → auto-deploy pipeline. Rollbacks are simply `git revert`.

Pillar 2: Security

Focus: Protect information, systems, and assets through risk assessments and mitigation strategies

Key Practices:

- **IAM:** Least-privilege policies, role-based access, no long-lived credentials
- **Encryption:** KMS for at-rest, TLS 1.2+ for in-transit, ACM for certificate management
- **Network:** Private subnets for workloads, VPC endpoints to avoid public internet, Security Groups + NACLs
- **Detection:** GuardDuty, Security Hub, CloudTrail, Config Rules

Example: In my Bedrock RAG architecture, I used VPC endpoints for S3, Bedrock, and OpenSearch to ensure **zero data traversal over the public internet**. All data encrypted with customer-managed KMS keys.

Pillar 3: Reliability

Focus: Ability of a system to recover from failures and meet demand

Key Practices:

- Multi-AZ deployments for databases and compute
- Auto Scaling groups with health checks
- Circuit breaker patterns for microservices
- Disaster Recovery: Pilot Light / Warm Standby / Multi-Region Active-Active

Example: For a 3-tier web application, I designed a multi-AZ EKS cluster with Pod Disruption Budgets, cross-region S3 replication, and RDS Multi-AZ with automated failover. RPO < 1 hour, RTO < 15 minutes.

Pillar 4: Performance Efficiency

Focus: Use computing resources efficiently to meet requirements and maintain efficiency as demand changes

Key Practices:

- Right-sizing EC2 instances using Compute Optimizer
- Caching strategies: ElastiCache (Redis), CloudFront CDN, API Gateway caching
- Serverless for variable workloads (Lambda, Fargate)
- Database selection: DynamoDB for key-value, Aurora for relational, OpenSearch for full-text search

Example: In a cost optimization exercise, I moved batch-processing workloads from always-on EC2 to Lambda + Step Functions, reducing compute costs by ~70%.

Pillar 5: Cost Optimization

Focus: Avoid unnecessary costs

Key Practices:

- Reserved Instances / Savings Plans for predictable workloads
- Spot Instances for fault-tolerant workloads (batch, CI/CD runners)
- S3 Lifecycle policies (Standard → IA → Glacier)
- Right-sizing with AWS Cost Explorer and Trusted Advisor
- **Eliminating NAT Gateway costs** using VPC endpoints

Example: In my Bedrock RAG project, I replaced NAT Gateway (\$32/month + data processing) with VPC endpoints, cutting networking costs by ~90%.

Pillar 6: Sustainability

Focus: Minimize environmental impact of cloud workloads

Key Practices:

- Use managed services (less idle compute)
 - Right-size resources
 - Use Graviton (ARM) instances for better performance-per-watt
 - Implement data lifecycle policies
-

Q4. How do you conduct an AWS Well-Architected Review for an existing workload?

Answer:

1. **Use the AWS Well-Architected Tool** in the AWS Console
2. **Define the workload** — name, description, environment, regions
3. **Answer pillar-specific questions** — the tool has ~50+ questions across 6 pillars
4. **Identify High-Risk Issues (HRIs)** — the tool flags architectural gaps
5. **Create an improvement plan** — prioritized by business impact and effort
6. **Track remediation** — assign owners, set deadlines, re-review quarterly

What I look for specifically:

- Single points of failure (no Multi-AZ? no auto-scaling?)
 - Missing encryption (at-rest and in-transit)
 - No disaster recovery plan
 - Over-provisioned resources (cost waste)
 - Missing monitoring/alerting
 - Hardcoded secrets (should use Secrets Manager/Parameter Store)
 - No IaC (manual console changes = drift risk)
-

Section 2: AWS Compute Services

Q5. Compare EC2, ECS (Fargate), EKS, and Lambda. When would you choose each?

Answer:

Criteria	EC2	ECS Fargate	EKS	Lambda
Control	Full OS control	Container-level	Container + orchestration	Function-level
Scaling	ASG (minutes)	Task-level (seconds)	Pod-level (seconds)	Request-level (ms)
Pricing	Per-instance-hour	Per vCPU/memory-second	EC2/Fargate + EKS fee (\$0.10/hr)	Per-invocation + duration
Ops Overhead	High (patching, AMIs)	Low	Medium-High	Minimal
Max Duration	Unlimited	Unlimited	Unlimited	15 minutes
Cold Start	N/A	~10-30s	N/A	~100ms-10s
Use Case	Legacy apps, GPU, custom OS	Containerized microservices	Complex orchestration, multi-cloud	Event-driven, APIs, glue code

My Decision Framework:

Is the workload event-driven and < 15 min?

YES → Lambda

NO → Is it containerized?

NO → EC2 (legacy, custom OS, GPU)

YES → Do you need advanced orchestration (service mesh, CRDs, Helm)?

YES → EKS

NO → ECS Fargate (simpler, less ops overhead)

Example from my experience: For the Bedrock RAG project, I chose **Lambda** for the query handler (event-driven, short-duration API calls) and **Lambda** for the S3 ingestion

pipeline (triggered by S3 events). If the workload had required long-running model training, I'd have used **ECS Fargate** or **EKS**.

Q6. Explain Auto Scaling strategies in AWS. How would you design auto-scaling for a web application?

Answer:

Types of Auto Scaling:

1. EC2 Auto Scaling Groups (ASG)

- **Target Tracking:** Maintain CPU at 60% → ASG adds/removes instances automatically
- **Step Scaling:** Add 2 instances when CPU > 70%, add 4 when > 90%
- **Scheduled:** Scale out at 9 AM, scale in at 6 PM for predictable traffic
- **Predictive:** ML-based, learns traffic patterns and pre-scales

2. Application Auto Scaling (for ECS, DynamoDB, Aurora, etc.)

- Same policies (target tracking, step, scheduled) but for non-EC2 resources

3. Kubernetes HPA/VPA/KEDA (EKS)

- **HPA (Horizontal Pod Autoscaler):** Scale pods based on CPU/memory/custom metrics
- **VPA (Vertical Pod Autoscaler):** Adjust pod resource requests/limits
- **KEDA:** Event-driven scaling (scale to zero, scale on SQS queue depth)
- **Cluster Autoscaler / Karpenter:** Scale nodes when pods are pending

Web Application Auto Scaling Design:

```
CloudFront (CDN – absorbs static content load)
  ↓
ALB (Application Load Balancer)
  ↓
EKS Cluster
  ├── HPA: Scale pods on CPU (target 60%) + custom metrics (requests/sec)
  └── Karpenter: Provision nodes on-demand (Spot for non-critical, On-Demand
```

```
for critical)
└─ PDB: Ensure minimum 2 pods always running
    ↓
Aurora (Auto Scaling read replicas based on CPU)
    ↓
ElastiCache Redis (cluster mode for sharding)
```

Key Design Decisions:

- **Cooldown periods:** 300s to prevent thrashing
 - **Scale-out fast, scale-in slow:** React quickly to load spikes, but don't prematurely kill instances
 - **Health check grace period:** 120s to let new instances warm up
 - **Mixed Instances Policy:** Use multiple instance types to improve Spot availability
-

Q7. What is AWS Lambda? Explain cold starts, concurrency, and when Lambda is NOT the right choice.

Answer:

AWS Lambda is a serverless compute service that runs code in response to events. You pay only for compute time consumed.

Cold Starts:

- Occurs when Lambda creates a new execution environment (download code, init runtime, run init code)
- **Python cold start:** ~200-500ms (relatively fast)
- **Mitigation strategies:**
 - **Provisioned Concurrency:** Pre-warm N environments (costs money but eliminates cold starts)
 - **Keep functions warm:** CloudWatch scheduled event every 5 minutes
 - **Minimize package size:** Use Lambda Layers, exclude unnecessary dependencies
 - **Use ARM (Graviton2):** 20% faster cold starts, 20% cheaper

Concurrency:

- **Account-level default:** 1,000 concurrent executions (can request increase)
- **Reserved Concurrency:** Guarantee N concurrent executions for a critical function
- **Provisioned Concurrency:** Pre-initialize N execution environments
- **Burst Concurrency:** 3,000 initial burst (us-east-1), then 500/minute

When Lambda is NOT the right choice:

- Execution > 15 minutes → Use ECS/Step Functions
- Needs persistent connections (WebSockets, long-polling) → Use ECS/EC2 + API Gateway WebSocket
- High and consistent traffic (millions of requests/sec) → EC2/ECS is more cost-effective
- GPU workloads → EC2 with GPU instances
- Large memory/compute needs > 10GB RAM → EC2/ECS
- Needs local filesystem persistence → ECS with EFS mount (Lambda has /tmp but limited)

Q8. Explain the difference between ECS and EKS. When would you recommend EKS over ECS?

Answer:

Aspect	ECS	EKS
Orchestrator	AWS proprietary	Kubernetes (CNCF standard)
Learning Curve	Lower	Higher
Portability	AWS-locked	Multi-cloud, on-prem
Service Mesh	AWS Cloud Map + App Mesh	Istio, Linkerd, AWS App Mesh
Custom Resources	Limited	CRDs, Operators, Helm charts
Cost	No control plane fee	0.10/hrpercluster(73/month)

Aspect	ECS	EKS
Ecosystem	AWS-native tools	Massive K8s ecosystem (ArgoCD, Prometheus, etc.)
Scaling	ECS Service Auto Scaling	HPA, VPA, KEDA, Karpenter

Recommend EKS when:

- Team already knows Kubernetes
- Multi-cloud or hybrid-cloud strategy
- Need advanced orchestration: CRDs, Operators, service mesh
- GitOps with ArgoCD/Flux
- Need to run on-prem + cloud (EKS Anywhere)
- Complex microservices with service-to-service mTLS

Recommend ECS when:

- Simpler workloads, fewer services
- Team is new to containers
- Want minimal ops overhead
- Fully AWS-native stack
- Cost-sensitive (no control plane fee)

My experience: I've worked extensively with EKS for production 3-tier applications — using Karpenter for node scaling, ArgoCD for GitOps deployments, and Terraform to provision the entire cluster. For simpler workloads, I'd recommend ECS Fargate to reduce operational burden.

Section 3: AWS Networking

Q9. Design a production VPC architecture for a 3-tier web application. Explain each component.

Answer:

Region: us-east-1
VPC: 10.0.0.0/16 (65,536 IPs)

AZ-1 (us-east-1a)

Public Subnet: 10.0.1.0/24 → ALB, NAT Gateway, Bastion

Private Subnet: 10.0.10.0/24 → Application (EKS worker nodes)

Data Subnet: 10.0.20.0/24 → RDS, ElastiCache

AZ-2 (us-east-1b)

Public Subnet: 10.0.2.0/24 → ALB, NAT Gateway

Private Subnet: 10.0.11.0/24 → Application (EKS worker nodes)

Data Subnet: 10.0.21.0/24 → RDS Standby, ElastiCache replica

AZ-3 (us-east-1c) [for EKS best practice: 3 AZs]

Public Subnet: 10.0.3.0/24

Private Subnet: 10.0.12.0/24

Data Subnet: 10.0.22.0/24

Components:

Component	Purpose	Placement
Internet Gateway	Internet access for public subnets	VPC-level
NAT Gateway	Outbound internet for private subnets	Public subnet (one per AZ for HA)
ALB	Layer 7 load balancing, SSL termination	Public subnets
EKS Worker Nodes	Application containers	Private subnets
RDS Multi-AZ	Relational database with auto-failover	Data subnets
VPC Endpoints	Private access to AWS services (S3, ECR, STS, etc.)	VPC-level
Security Groups	Stateful instance-level firewall	Per resource
NACLs	Stateless subnet-level firewall	Per subnet
Route Tables	Traffic routing rules	Per subnet tier

Security Group Rules (layered):

ALB SG:	Inbound 443 from 0.0.0.0/0
App SG:	Inbound 8080 from ALB SG only
DB SG:	Inbound 5432 from App SG only

Cost Optimization: Replace NAT Gateway with VPC endpoints for AWS service access (S3, ECR, CloudWatch, Bedrock) — saves \$32+/month per NAT Gateway plus data processing charges.

Q10. Explain VPC Endpoints — Interface vs Gateway. Why are they important?

Answer:

VPC Endpoints provide **private connectivity** from your VPC to AWS services without traversing the public internet.

Gateway Endpoints (Free):

- Supported services: **S3** and **DynamoDB** only
- Implemented as a route table entry
- No ENI, no security group — traffic stays within AWS network
- Free to use (no hourly or data processing charge)

Interface Endpoints (Powered by AWS PrivateLink):

- Supported services: ~100+ services (ECR, CloudWatch, Bedrock, STS, KMS, etc.)
- Creates an **ENI** in your subnet with a private IP
- Can attach Security Groups for access control
- Costs: ~0.01/hr per AZ + 0.01/GB data processed

Why Important:

1. **Security:** Traffic never leaves AWS backbone network → no exposure to internet
2. **Cost:** Eliminates NAT Gateway data processing charges (\$0.045/GB)
3. **Compliance:** Required for workloads in private subnets with no internet access
4. **Performance:** Lower latency (no NAT hop)

Example from my Bedrock RAG project:

```
# VPC Endpoints I configured (Terraform):  
- com.amazonaws.us-east-1.s3           (Gateway - free)  
- com.amazonaws.us-east-1.bedrock-runtime (Interface)  
- com.amazonaws.us-east-1.execute-api   (Interface)  
- com.amazonaws.us-east-1.logs          (Interface)  
- com.amazonaws.us-east-1.kms           (Interface)
```

This eliminated the need for a NAT Gateway entirely, keeping all traffic private and reducing costs.

Q11. Explain Route53 routing policies and when to use each.

Answer:

Routing Policy	Use Case	How It Works
Simple	Single resource	Returns one or more IPs (round-robin if multiple)
Weighted	A/B testing, blue/green, canary	Assign weights (e.g., 90% to v1, 10% to v2)
Latency	Multi-region apps	Routes to region with lowest latency for user
Failover	DR/HA	Primary → Health Check fails → Secondary
Geolocation	Compliance, localization	Route based on user's geographic location
Geoproximity	Fine-grained traffic shifting	Route based on location + bias value to shift traffic
Multi-Value Answer	Simple load balancing with health checks	Returns up to 8 healthy records

Real-World Architecture Example — Multi-Region Active-Passive:

```
api.myapp.com
├─ Failover Primary:  us-east-1 ALB (health check: /health)
└─ Failover Secondary: ap-south-1 ALB (health check: /health)
```

With Weighted for Canary Deployments:

```
api.myapp.com
├─ Weight 95: ALB-v1 (current production)
└─ Weight 5:  ALB-v2 (canary release)
```

Q12. What is CloudFront and how does it integrate with your architecture?

Answer:

CloudFront is AWS's Content Delivery Network (CDN) with 450+ Points of Presence (PoPs) globally.

Architecture Integration:

```
User → CloudFront (Edge) → Origin (ALB / S3 / API Gateway)
      ↓
    Edge Functions:
    └─ CloudFront Functions (viewer request/response)
    └─ Lambda@Edge (origin request/response)
```

Key Features I Use:

1. **Static Content Caching:** S3 origin for React/Angular SPA bundles, images, CSS/JS
2. **Dynamic Content Acceleration:** ALB origin for API calls (TCP optimization, keep-alive)
3. **SSL/TLS Termination:** ACM certificate at the edge, HTTP/2, TLS 1.3
4. **Security:**
 - AWS WAF integration (rate limiting, SQL injection, XSS protection)
 - Origin Access Control (OAC) for S3 — blocks direct S3 access
 - Geo-restriction for compliance

5. Cache Behaviors:

- `/api/*` → Forward to ALB, no caching (or short TTL)
- `/static/*` → S3 origin, cache for 1 year (content-hashed filenames)
- `/*` → S3 origin, cache for 24 hours

Cache Invalidation Strategy:

- Use content-hashed filenames (`main.a1b2c3.js`) — never need invalidation
- For HTML files: short TTL (5 minutes) or invalidation via CI/CD pipeline
- `aws cloudfront create-invalidation --paths "/*"` — costs \$0.005 per path after first 1,000 free

Q13. Explain API Gateway — REST vs HTTP vs WebSocket. When would you use each?

Answer:

Feature	REST API	HTTP API	WebSocket API
Cost	\$3.50/million	\$1.00/million	\$1.00/million + connection minutes
Latency	Higher	~60% lower	Persistent connection
Features	Full (caching, WAF, usage plans, API keys, request validation)	Minimal (JWT auth, CORS, OIDC)	Real-time bi-directional
Auth	IAM, Cognito, Lambda authorizer, API keys	IAM, JWT, Lambda authorizer	IAM, Lambda authorizer
Use Case	Enterprise APIs needing throttling, caching, documentation	High-perf, cost-sensitive APIs	Chat, notifications, live updates

My Decision:

- **REST API** when I need: WAF integration, request/response transformation, API key management, caching, detailed CloudWatch metrics

- **HTTP API** when I need: Simple proxy to Lambda/ALB, low latency, cost savings
- **WebSocket** when I need: Real-time communication (chat, live dashboards)

Example: In my Bedrock RAG project, I used **REST API** with:

- Lambda proxy integration for the query endpoint
 - IAM authorization
 - Request validation
 - Usage plan with throttling (100 req/sec steady, 200 burst)
 - Stage variables for dev/staging/prod environments
-

This concludes Part 1. Continue to Part 2 for Security, Storage, Database, and IaC sections.